

Procesamiento del Lenguaje Natural I

TEMA 5. TRANSFORMERS (I)

Soto Montalvo Herranz

Objetivos

- Conocer los fundamentos de los Transformers.
- Aprender a utilizarlos para tareas de PLN.
- Conocer las limitaciones y desafíos de los Transformers.

Índice

- Conceptos preliminares
- ¿Qué son los Transformers?
- Arquitectura de los Transformers
- ¿Cómo se trabaja con los Transformers?
- Limitaciones y desafíos de los Transformers en PLN
- Bibliografía

Índice

- **Conceptos preliminares**
- ¿Qué son los Transformers?
- Arquitectura de los Transformers
- ¿Cómo se trabaja con los Transformers?
- Limitaciones y desafíos de los Transformers en PLN
- Bibliografía

Conceptos preliminares

- Es momento de recordar la evolución del PLN en los últimos años:
 - Hace 12 o 13 años se utilizaba habitualmente BoW como entrada de los algoritmos de ML.
 - En la última década explotó el DL e irrumpieron los embeddings => el propio texto pasaba a ser la entrada de los modelos
 - RNN se convirtieron en el estándar, pero, tenían memoria a corto plazo.
 - Ejemplo: generación de texto (en color rojo el texto generado de forma incorrecta).

[illegible]

Imagen: [4]

Conceptos preliminares

- ¿Cuándo aparecieron los Transformers?
 - En el año 2017, en un paper de Google titulado “Attention is All You Need” [3] se presentó la arquitectura de Transformers.
- ¿Cuál es la innovación en los Transformers? Sustituyen las capas recurrentes por las **capas de atención**.
- Las capas de atención codifican cada palabra de una frase en función del resto de la secuencia, permitiendo así **introducir el contexto en la representación matemática del texto**
 - A los modelos basados en Transformer se les denomina también **Embeddings Contextuales**.
 - La arquitectura de Transformer también incluye los **embeddings posicionales**, que permiten al algoritmo conocer la posición relativa de cada palabra del texto.
- Los Transformers son la arquitectura estándar para construir modelos de lenguaje.

Índice

- Conceptos preliminares
- **¿Qué son los Transformers?**
- Arquitectura de los Transformers
- ¿Cómo se trabaja con los Transformers?
- Limitaciones y desafíos de los Transformers en PLN
- Bibliografía

¿Qué son los Transformers?

- Son una arquitectura de red neuronal que se utiliza para Procesamiento del Lenguaje Natural.
- No requieren una entrada secuencial => son capaces de procesar palabras y oraciones completas **en paralelo**.
 - Lo logran por su capacidad de capturar las dependencias contextuales a largo plazo mediante los mecanismos de atención.
- Aprenden el contexto de los datos secuenciales y son capaces de generar nuevos datos a partir de él.
- El **mecanismo de atención** es el **componente clave** en los Transformers

Índice

- Conceptos preliminares
- ¿Qué son los Transformers?
- **Arquitectura de los Transformers**
- ¿Cómo se trabaja con los Transformers?
- Limitaciones y desafíos de los Transformers en PLN
- Bibliografía

Arquitectura de los Transformers (previo)

- La arquitectura de un Transformer es una evolución de la arquitectura *Encoder-Decoder*.
- El modelo Seq2Seq ha alcanzado mucho éxito en tareas como traducción automática. Está compuesto por:
 - Un codificador (*encoder*), que recibe la secuencia de entrada y genera una representación intermedia
 - Un decodificador (*decoder*), que recibe la salida del codificador y produce la secuencia de salida.

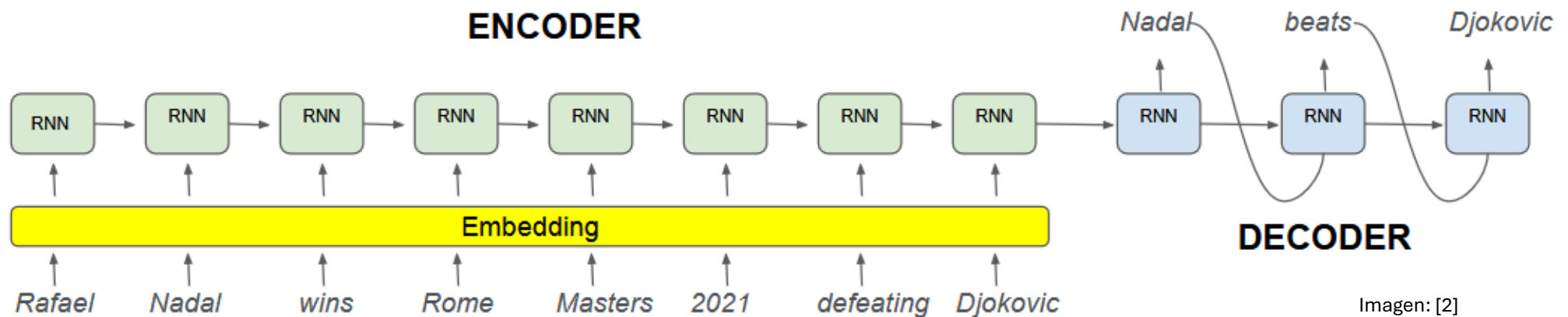


Imagen: [2]

Arquitectura de los Transformers (previo)

- Limitaciones del modelo Seq2Seq:
 - La salida se basa, en gran medida, en el contexto definido por el estado oculto de la salida final del codificador, esto dificulta el manejo de oraciones largas (mucho de la información de los primeros tokens puede que ya no esté representada en el vector que produce el codificador).
 - No permite paralelización.
- ¿Cómo solucionar estas limitaciones?
 - Se añade al modelo Seq2Seq un **mecanismo de atención**.
 - El mecanismo de atención permite generar cada token en la secuencia de salida en función de los tokens más relevantes en la oración para dicho token.
 - El codificador genera un vector de salida por cada token de entrada.

Arquitectura de los Transformers (previo)

- Mecanismo de atención:

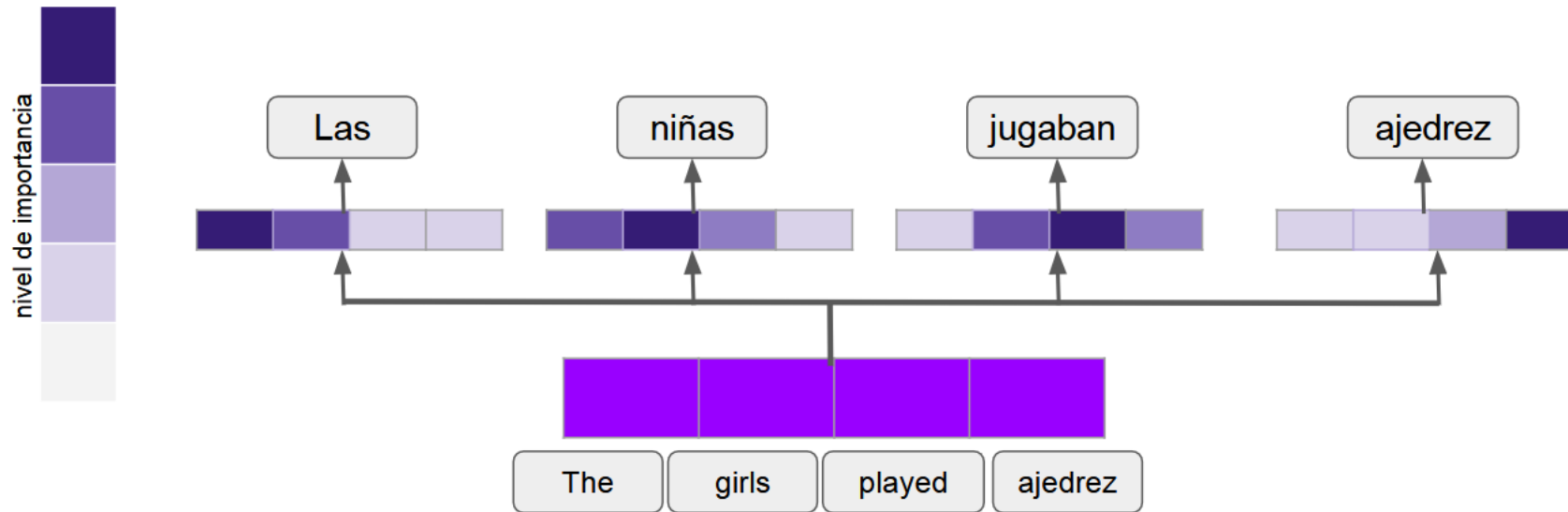
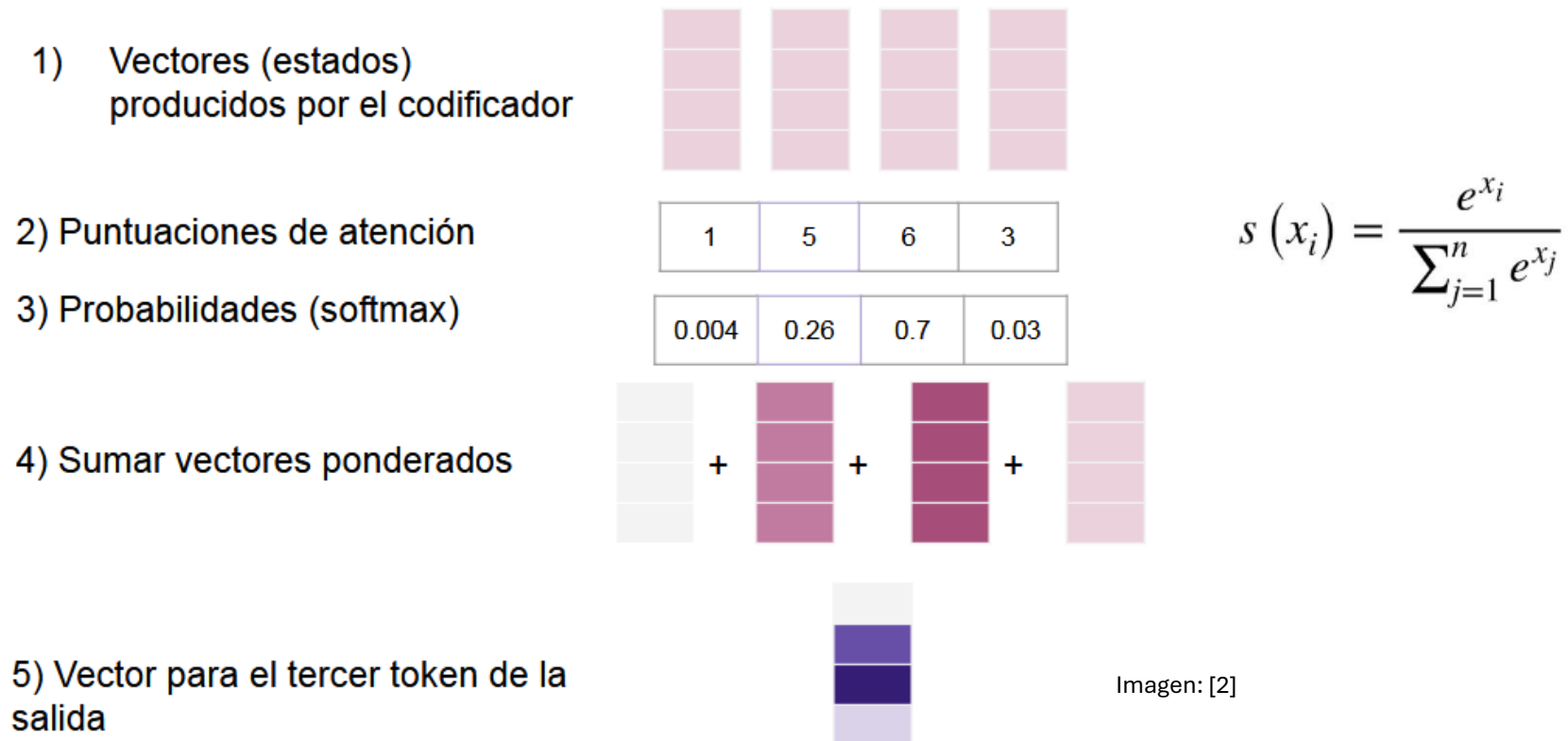


Imagen: [2]

Arquitectura de los Transformers (previo)

- Ejemplo para generar el tercer token de salida (“jugaban”):



Arquitectura de los Transformers (previo)

- Ventajas del mecanismo de atención:
 - Ya no se pierde la información en secuencias largas como ocurría con las RNN, ahora todos los estados del codificador se pasan al decodificador.
 - El decodificador puede aprender qué partes de la entrada son más relevantes, es decir, puede aprender qué estados ocultos generados por el codificador son más relevantes en cada paso en la secuencia de salida.
- Limitaciones del mecanismo de atención:
 - Procesamiento secuencial, se procesa un elemento cada vez => cada estado oculto depende de la salida del estado oculto anterior. No se puede empezar el paso t , hasta que no se ha terminado el paso $t-1$.
 - Esto es ineficiente en conjuntos de datos grandes.
- ¿Entonces?

Arquitectura de los Transformers (primer modelo)

- Vaswani, A. et al. (2017). *Attention is all you need*. Advances in neural information processing systems, 30.

No recurrence. No convolutions. Just **attention**.

- La arquitectura de los transformers se apoya en varios conceptos:
 - *Positional encoding*: las palabras se numeran según su posición en la frase => el modelo puede conocer el orden de las palabras.
 - *Self-attention*: cuando se procesa un token de la secuencia de entrada, el codificador identifica qué otros tokens de la secuencia son los más relevantes para dicho token.

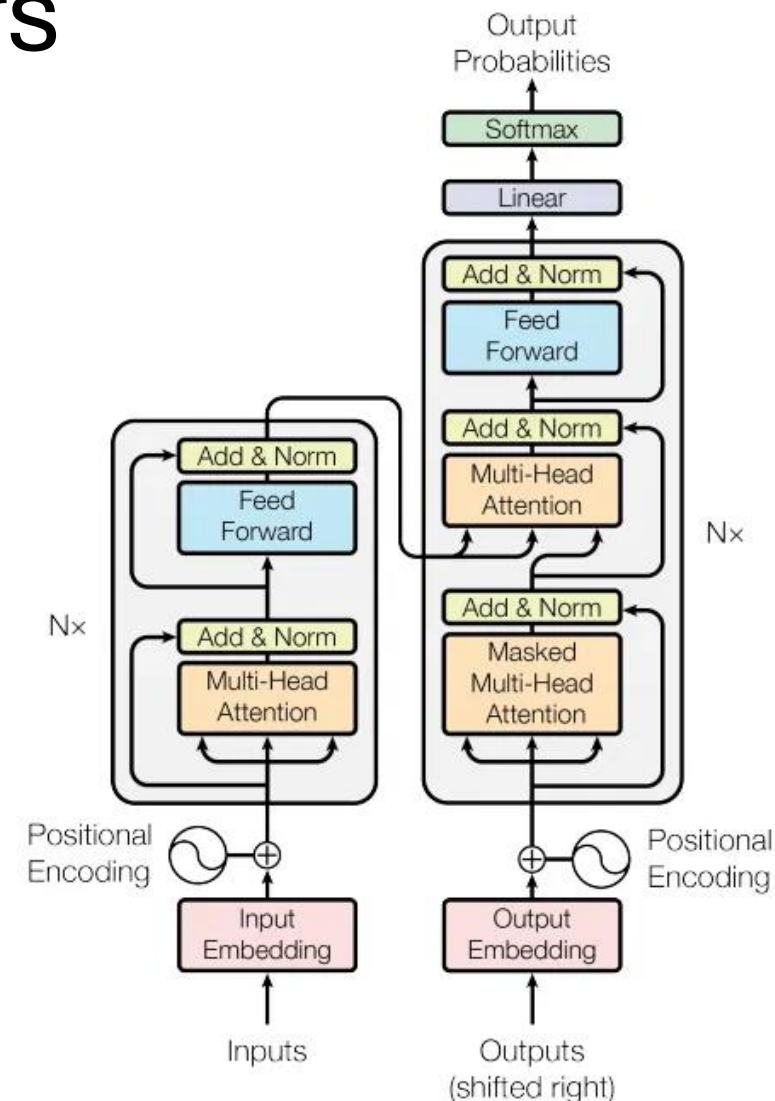


Imagen: [3]

Arquitectura de los Transformers (primer modelo)

- Los codificadores y decodificadores leen la información, la procesan y devuelven los resultados.
- El mecanismo de atención no permitía la paralelización.
- El mecanismo de *self-attention* sí permite paralelización (*multi-head*).

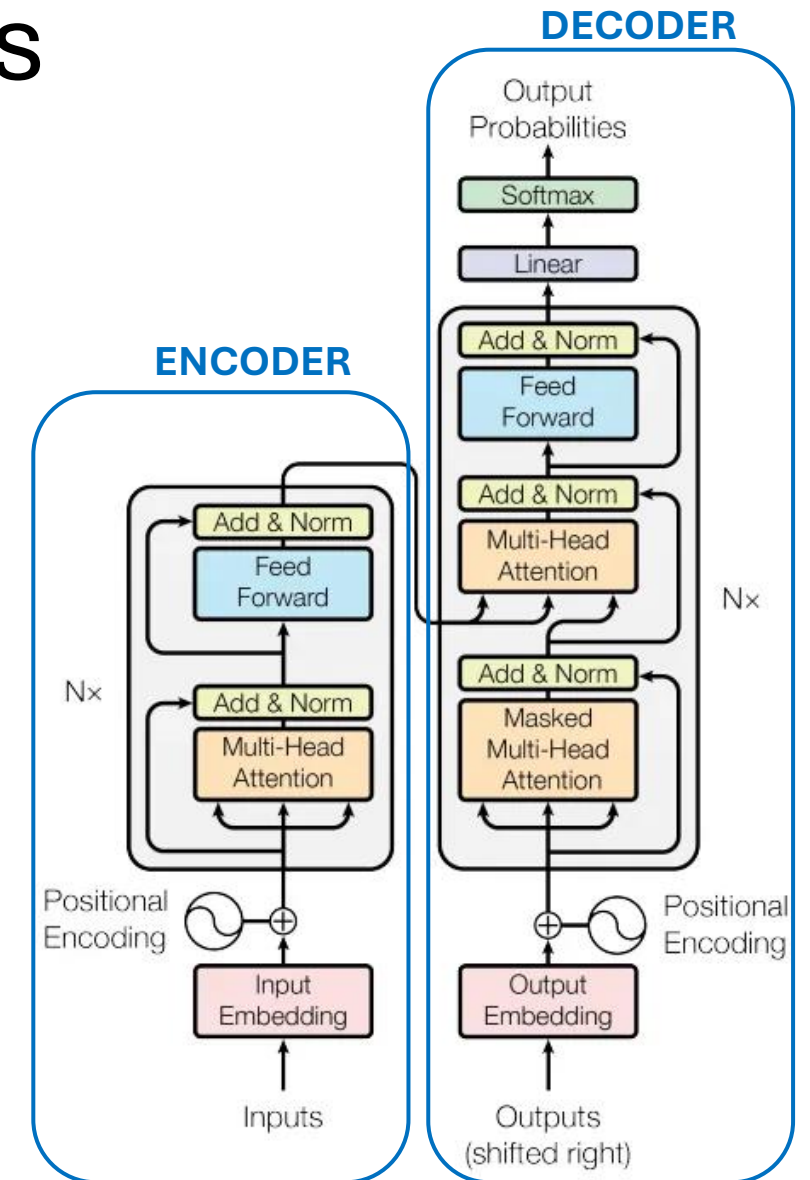
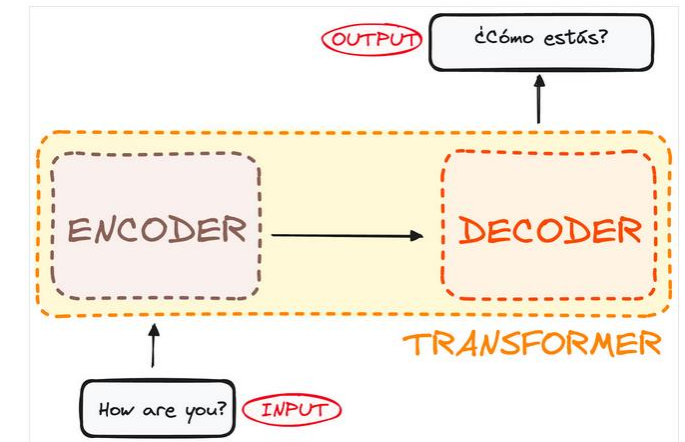


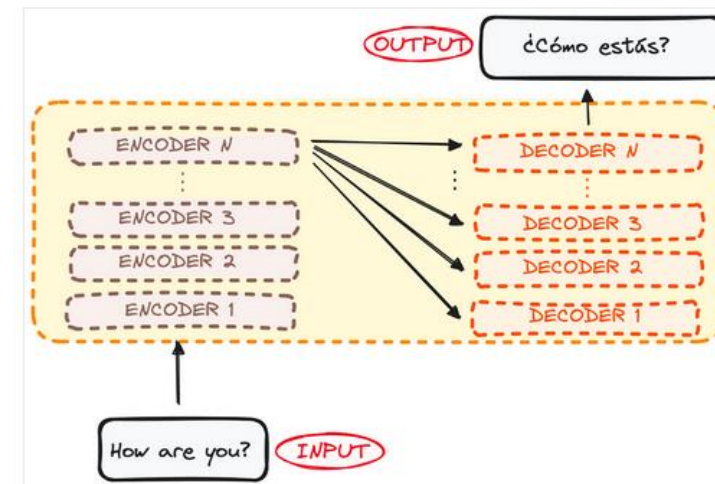
Imagen: [3]

Arquitectura de los Transformers

- Se mantiene el modelo *encoder-decoder*.
- En realidad, el codificador y decodificador son pilas con varias capas (mismo número en cada uno)
 - En los codificadores entra la entrada y pasa al siguiente codificador.
 - En los decodificadores se recibe como entrada la del último codificador y del decodificador anterior.



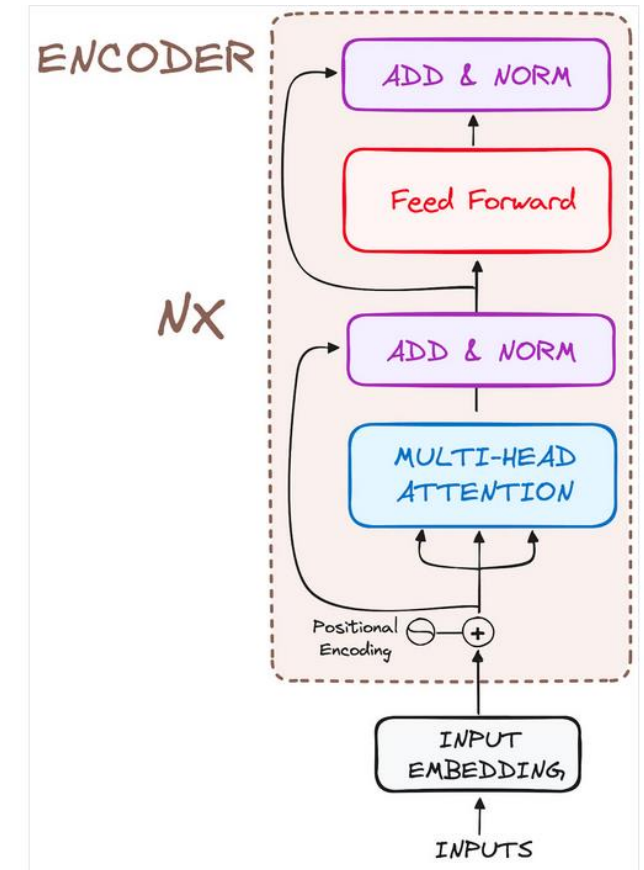
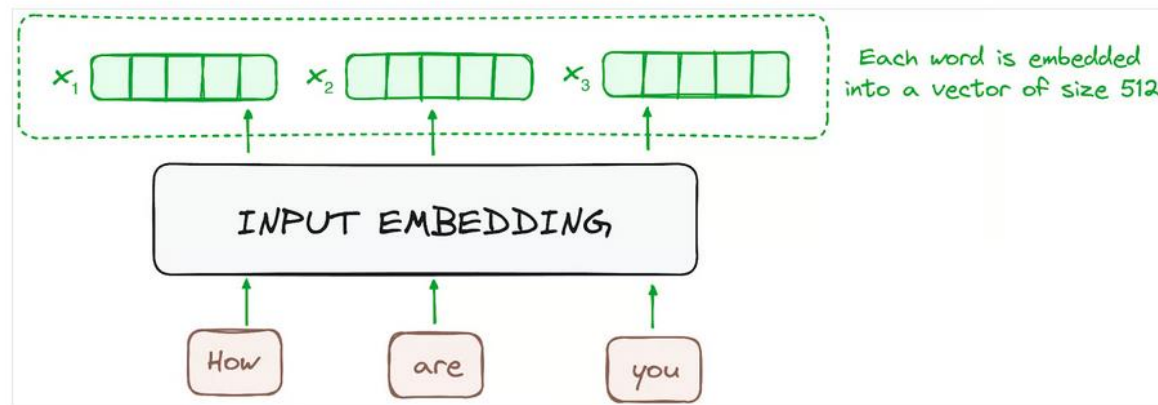
Ejemplo suponiendo N capas



Imágenes: [5]

Arquitectura de los Transformers

- ¿Qué hace el codificador (*Encoder*)?
 - Transforma los tokens de entrada en representaciones contextualizadas => capta el contexto de cada token con respecto a la secuencia.
 - Las tareas del codificador son varias:
 1. Convierte la entrada en vectores de embeddings.



Imágenes: [5]

Arquitectura de los Transformers

- ¿Qué hace el codificador (*Encoder*)? Tareas:
 2. Codificación posicional (*Positional encoding*)

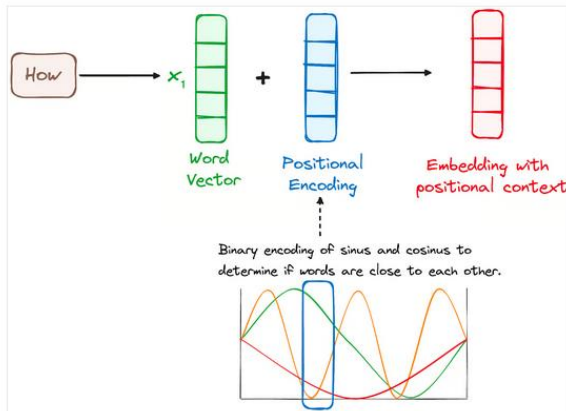


Imagen: [5]

Se combinan funciones seno y coseno para crear vectores posicionales.

Se producen codificaciones únicas para cada posición.

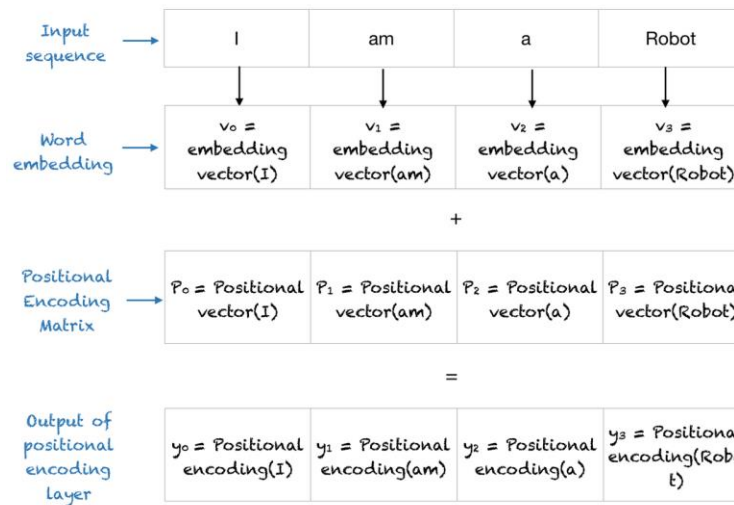


Imagen: [6]

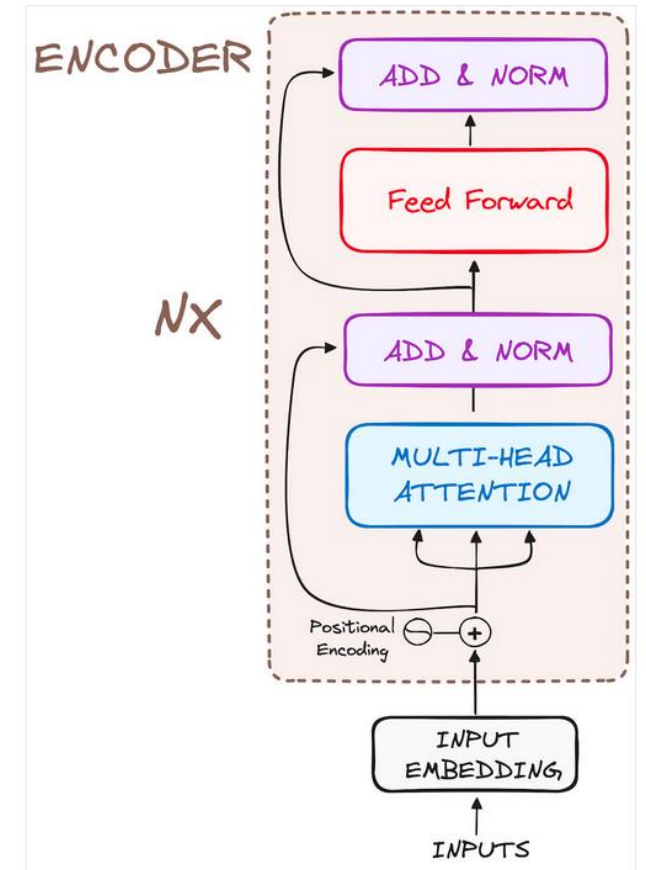


Imagen: [5]

Arquitectura de los Transformers

- ¿Qué hace el codificador (*Encoder*)? Tareas:
 - Codificación posicional (*Positional encoding*)

Sequence	Index of token	Positional Encoding Matrix			
I	0	P_{00}	P_{01}	...	P_{0d}
am	1	P_{10}	P_{11}	...	P_{1d}
a	2	P_{20}	P_{21}	...	P_{2d}
Robot	3	P_{30}	P_{31}	...	P_{3d}

Positional Encoding Matrix for the sequence 'I am a robot'

Sequence	Index of token, k	Positional Encoding Matrix with $d=4, n=100$			
		$i=0$	$i=0$	$i=1$	$i=1$
I	0	$P_{00}=\sin(0)=0$	$P_{01}=\cos(0)=1$	$P_{02}=\sin(0)=0$	$P_{03}=\cos(0)=1$
am	1	$P_{10}=\sin(1/10)=0.84$	$P_{11}=\cos(1/10)=0.54$	$P_{12}=\sin(1/10)=0.10$	$P_{13}=\cos(1/10)=1.0$
a	2	$P_{20}=\sin(2/10)=0.91$	$P_{21}=\cos(2/10)=-0.42$	$P_{22}=\sin(2/10)=0.20$	$P_{23}=\cos(2/10)=0.98$
Robot	3	$P_{30}=\sin(3/10)=0.14$	$P_{31}=\cos(3/10)=-0.99$	$P_{32}=\sin(3/10)=0.30$	$P_{33}=\cos(3/10)=0.96$

Positional Encoding Matrix for the sequence 'I am a robot'

$$P(1,0) = \sin\left(\frac{1}{10000^{0/4}}\right) = \sin(1) \approx 0.84$$

$$P(1,1) = \cos\left(\frac{1}{10000^{0/4}}\right) = \cos(1) \approx 0.54$$

Imágenes: [6]

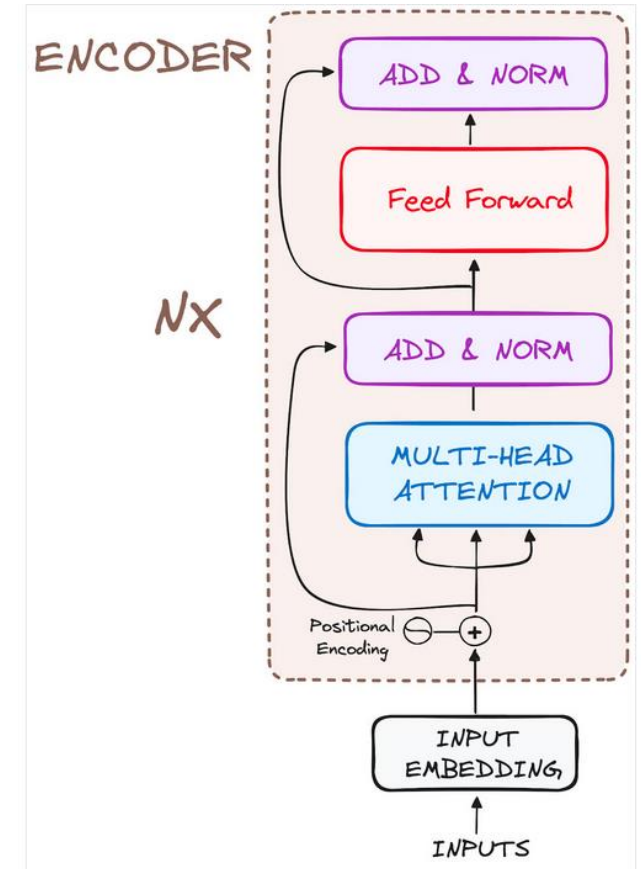
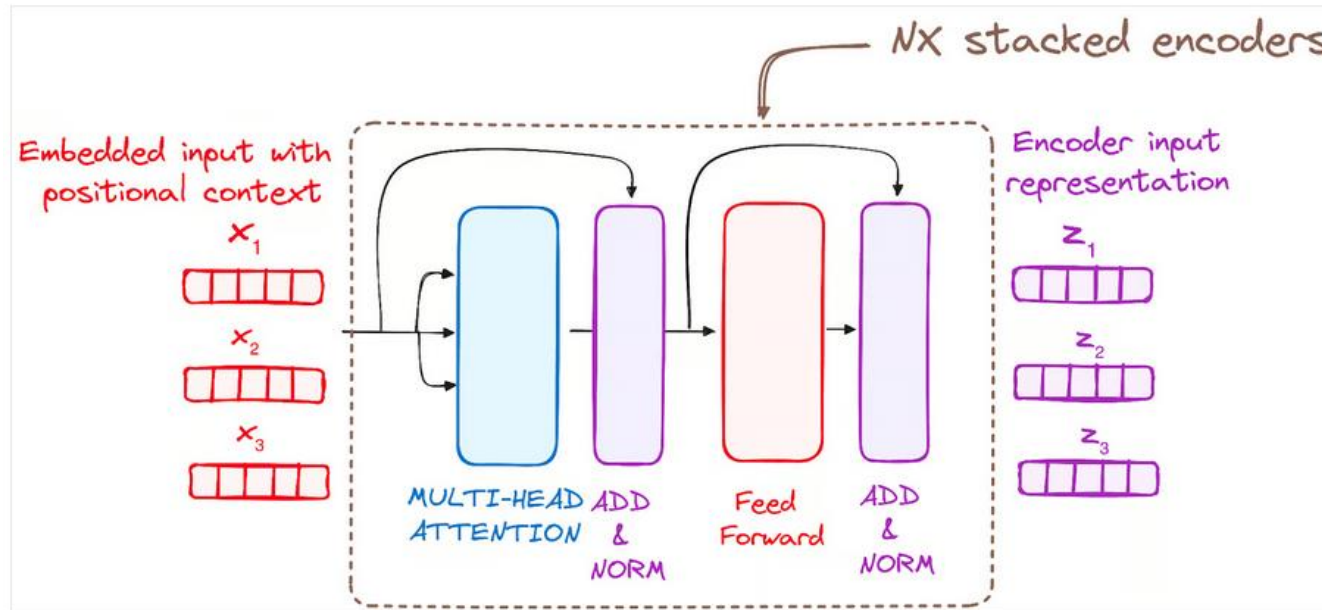
$$P(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right)$$

$$P(k, 2i+1) = \cos\left(\frac{k}{n^{2i/d}}\right)$$

- k : Position of an object in the input sequence, $0 < k < L/2$
- d : Dimension of the output embedding space
- $P(k, j)$: Position function for mapping a position k in the input sequence to index (k, j) of the positional matrix
- n : User-defined scalar, set to 10,000 by the authors of Attention Is All You Need.
- i : Used for mapping to column indices $0 < i < d/2$, with a single value of i maps to both sine and cosine functions

Arquitectura de los Transformers

- ¿Qué hace el codificador (*Encoder*)? Tareas:
 3. Pila de capas del codificador



Imágenes: [5]

Arquitectura de los Transformers

- ¿Qué hace el codificador (*Encoder*)? Tareas:
3.1. *Self-Attention (Multi-Head Attention)*

Se relaciona cada palabra de la entrada con otras palabras de la secuencia.

Las puntuaciones de atención se calculan utilizando tres representaciones alternativas de los tokens de entrada. Cada embedding de la entrada se transforma en tres vectores:

- Consulta (query): define qué busca el token en otros tokens
- Clave (key): indica cómo se define cada token a sí mismo
- Valor (value): define qué da cada token a otros tokens
 - Si la similitud de los vectores de la consulta y la clave es alta, al multiplicarse por el vector del valor, quedará resaltado en la salida el score final viendo si las palabras están más o menos relacionadas.

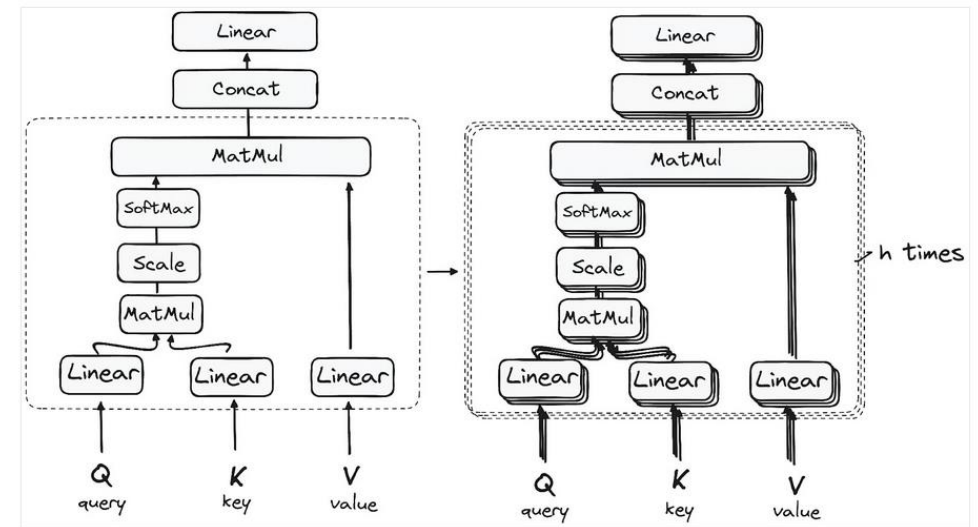


Imagen: [5]

Arquitectura de los Transformers

- ¿Qué hace el codificador (*Encoder*)? Tareas:
3.1. Self-Attention (Multi-Head Attention).

Query (Q): lo que se busca con una palabra.

Key (K): cómo se describe a si misma la palabra.

Value (V): información que aporta la palabra si se le presta atención.

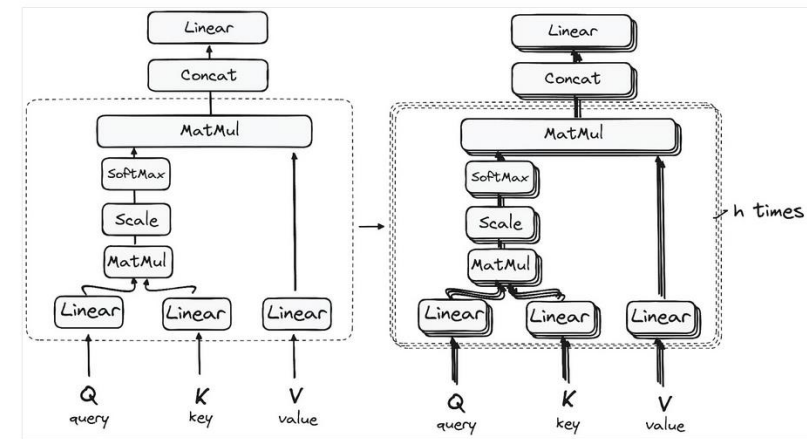


Imagen: [5]

Ejemplo [7]: “**The girl dropped her book because she was tired.**” Se necesita saber a quién se refiere “she”

La Query (Q) para “she” pregunta “¿A quién me refiero?” (*Who am I referring to?*)

El modelo compara esta consulta con las claves (K) de todas las demás palabras de la frase.

La clave de “girl” coincide más que la de “book”, entonces el valor (V) de “girl” contribuye más a cómo el modelo entiende “she”.

Arquitectura de los Transformers



Imagen: [4]

Las puntuaciones se reducen dividiéndolas por la raíz cuadrada de la dimensión de los vectores de consulta y clave.

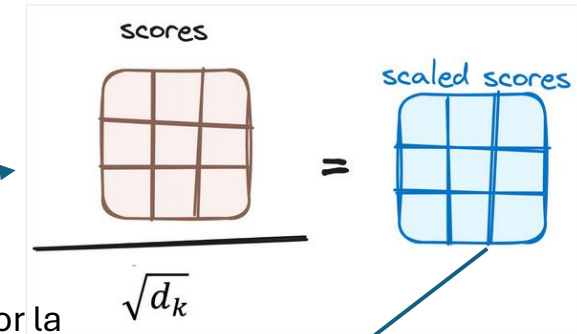


Imagen: [5]

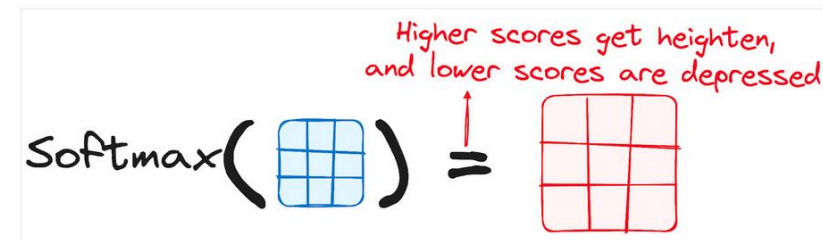
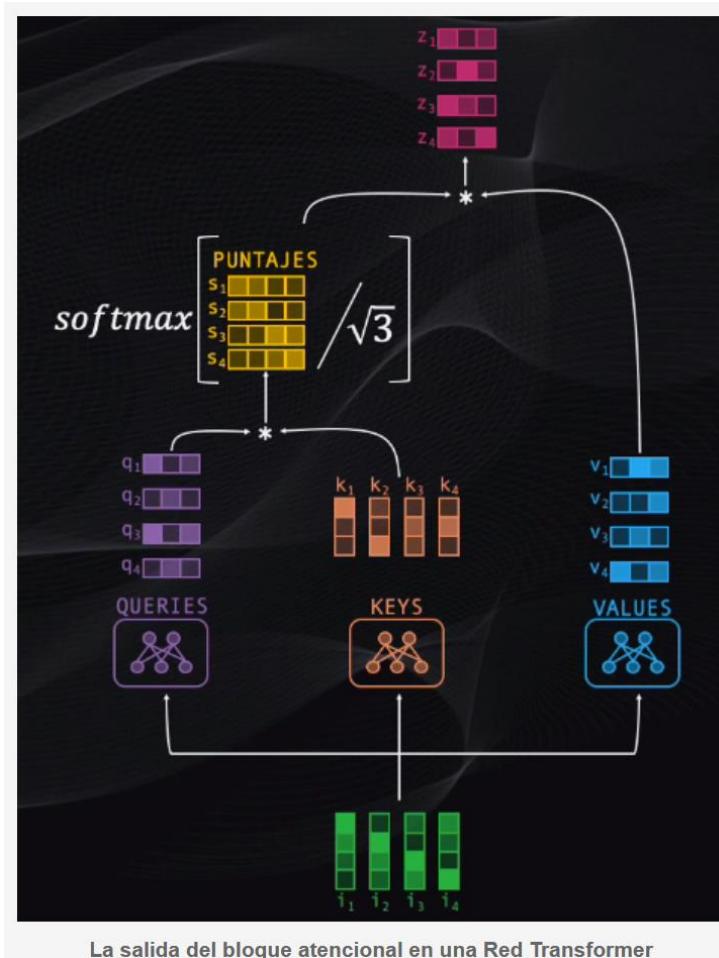


Imagen: [5]

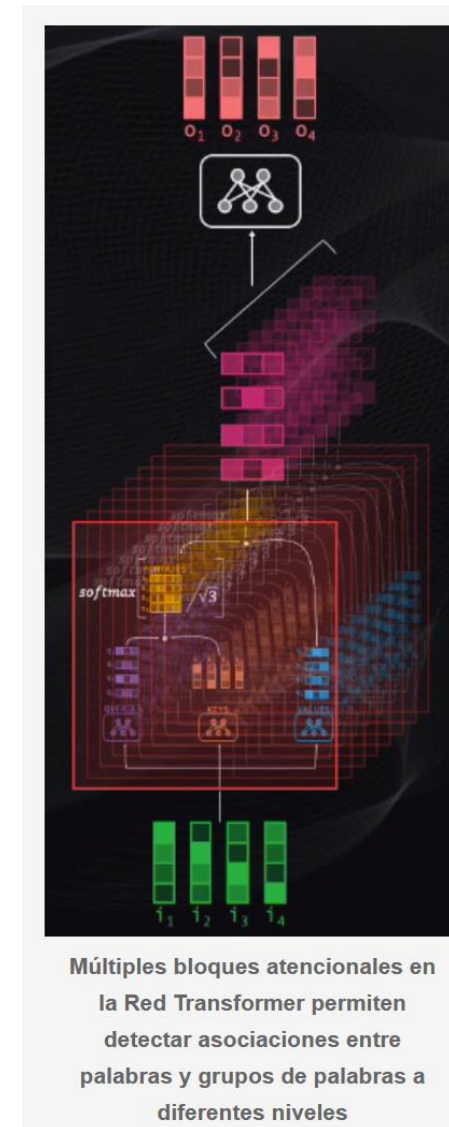
Al aplicar softmax se obtienen valores entre 0 y 1. Enfatiza las puntuaciones más altas al tiempo que disminuye las más bajas => ayuda al modelo a que sepa qué palabras tienen que recibir más atención.

Arquitectura de los Transformers



- Multi-Head porque el mecanismo de autoatención ocurre por separado en múltiples etapas o “cabezas”, cada una ocurre de forma independiente.
- Al usar múltiples capas, se pueden detectar asociaciones entre palabras y entre grupos de palabras a diferentes niveles.
- Las salidas de estas capas se combinan en una red neuronal que condensa toda la información resultante en un único vector para cada token de entrada.

Imágenes: [4]



Arquitectura de los Transformers

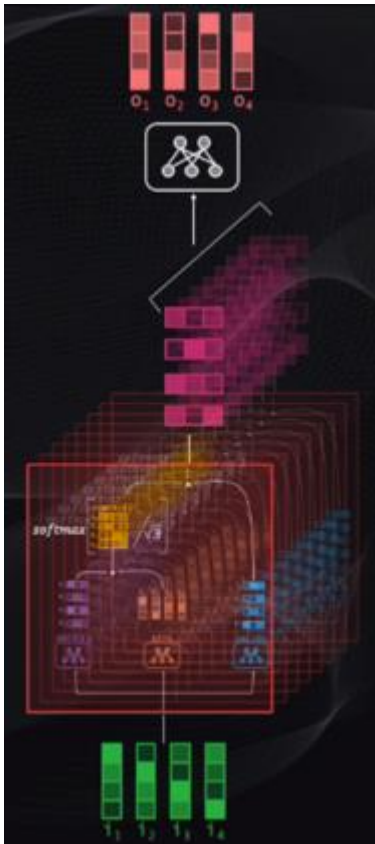


Imagen: [4]

Siguiendo con el ejemplo de la transparencia 22:

“The girl dropped her book because she was tired.”

¿Qué podrían interpretar cuatro lectores diferentes?

Head 1: Tracks subject-verb connections (“girl” → “dropped”)

Head 2: Resolves pronouns (“she” → “girl”)

Head 3: Focuses on cause-and-effect (“because” → “tired”)

Head 4: Identifies object ownership (“her” → “book”)

Cada lector (cabeza de atención) percibe algo diferente => al combinar sus percepciones, el modelo comprende la frase con mayor profundidad.

Arquitectura de los Transformers

- ¿Qué hace el codificador (*Encoder*)? Tareas:

3.2. Normalización y conexión residual

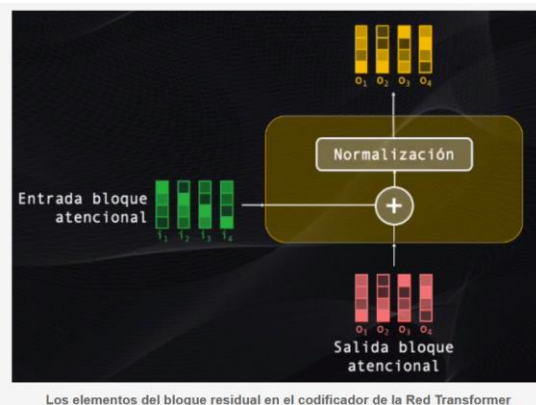
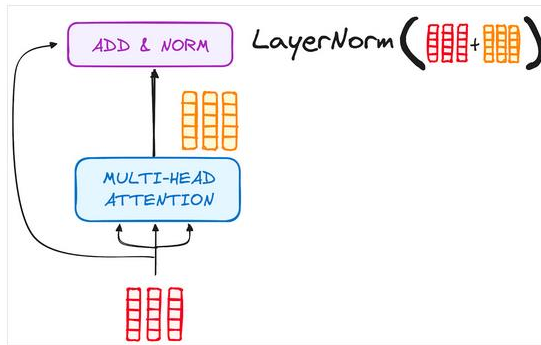


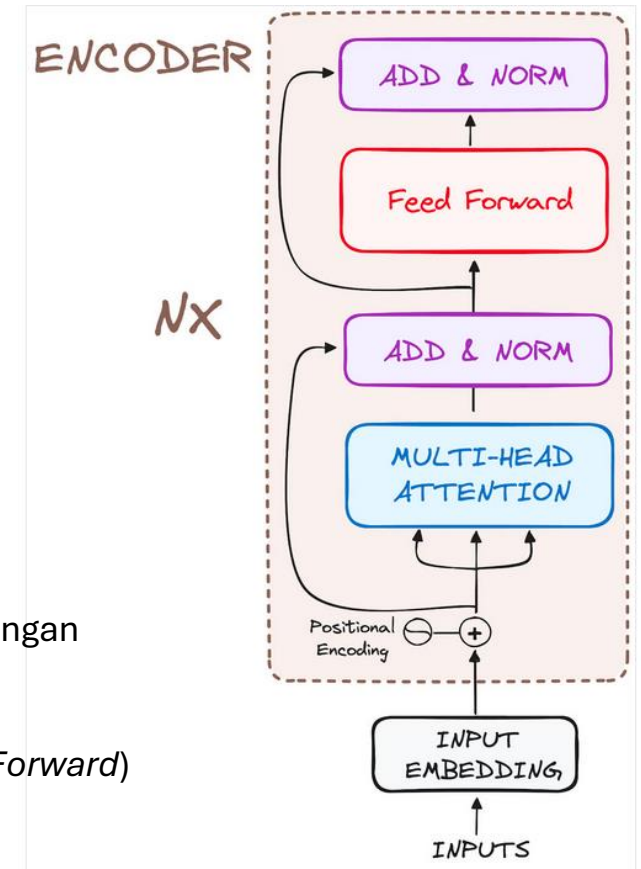
Imagen: [4]

Cada subcapa va seguida de una capa de normalización.

La salida de cada subcapa se añade a su entrada (conexión residual), para ayudar a mitigar el problema del descenso del gradiente.

Se toman los datos, se suman y se normalizan para que tengan la escala adecuada para la siguiente capa.

(Se repite todo también después de la red neuronal *Feed Forward*)



Imágenes: [5]

Arquitectura de los Transformers

- ¿Qué hace el codificador (*Encoder*)? Tareas:
3.3. Red Neuronal Feed Forward.

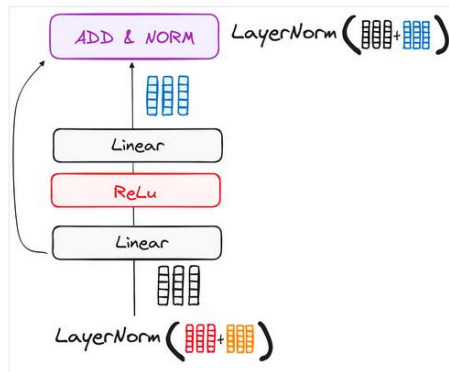


Imagen: [5]

Ejemplo de red neuronal con dos capas lineales y una activación ReLU entre ellas.

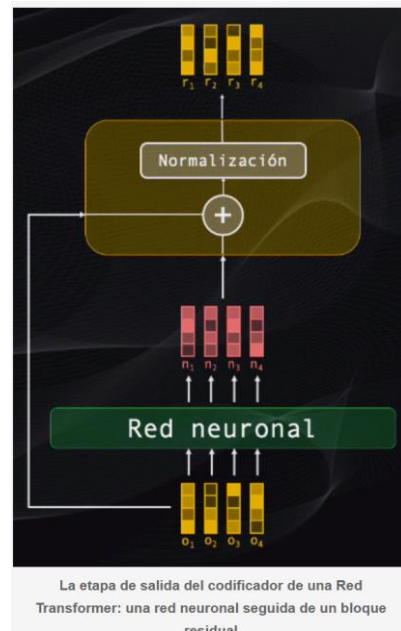


Imagen: [4]

En definitiva, se tiene una red neuronal con otra capa de normalización y conexión residual.

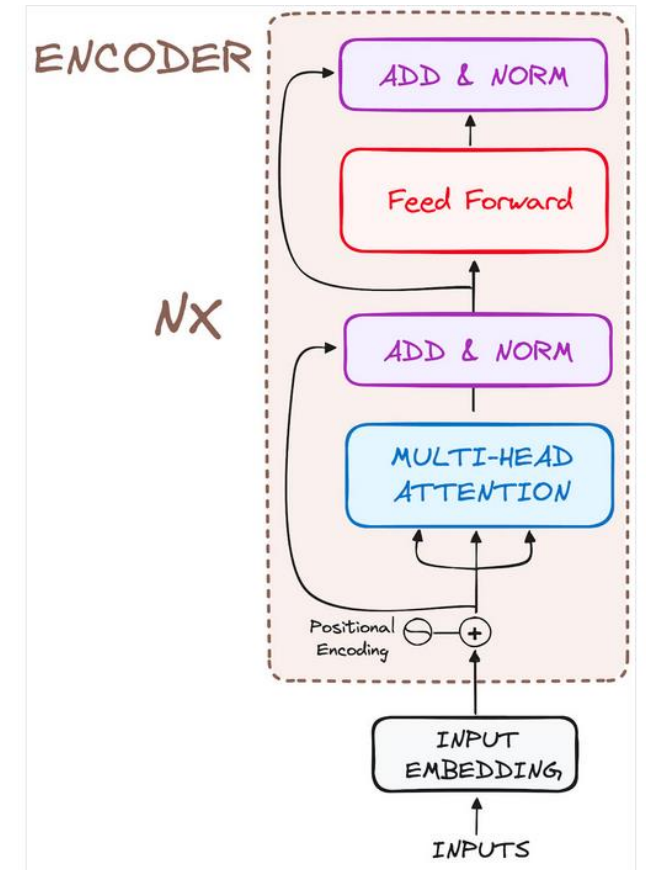


Imagen: [5]

Arquitectura de los Transformers

- ¿Qué hace el codificador (*Encoder*)? Tareas:
 4. Salida del codificador



Imagen: [4]

La salida es un conjunto de vectores, que se utilizará como entrada:

- Para otro codificador (puede haber varios en el Transformer)
- Para los decodificadores (puede haber varios)

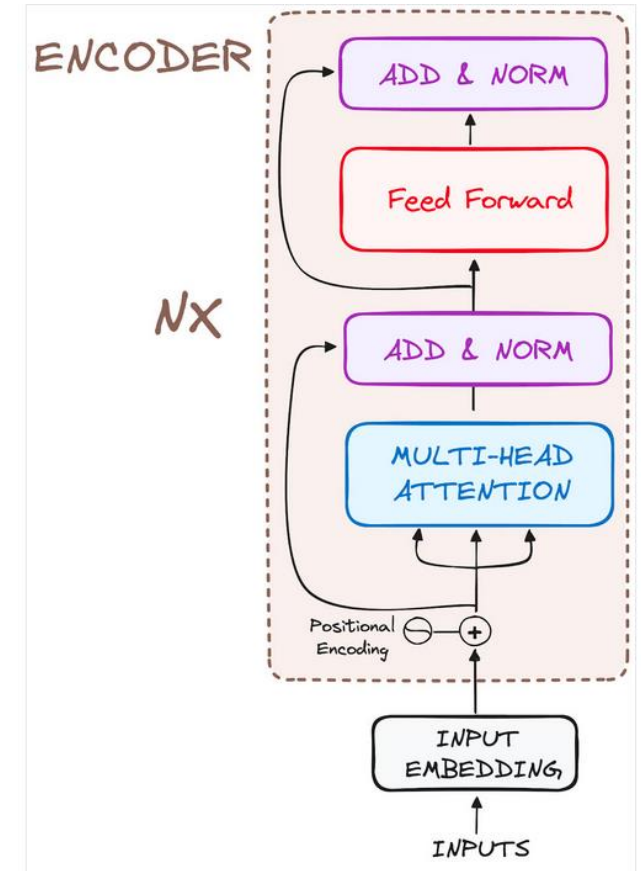


Imagen: [5]

Arquitectura de los Transformers

- ¿Qué hace el decodificador (*Decoder*)?
 - Genera secuencias de texto.
 - Las tareas del decodificador son varias:
 1. Generar los embeddings de la entrada que recibe.
 2. Añadir la información de la posición.
 3. Hay diferentes capas, donde cada una tiene tres componentes (entre ellos capas de normalización):
 - Mecanismo de autoatención enmascarada (*Masked Multi-Head Attention*)
 - Atención Multicabezal (*Multi-Head Attention*)
 - Red Neuronal *Feed-Forward*
 4. Clasificador lineal y Softmax, permite generar una a una las palabras de la secuencia de salida.

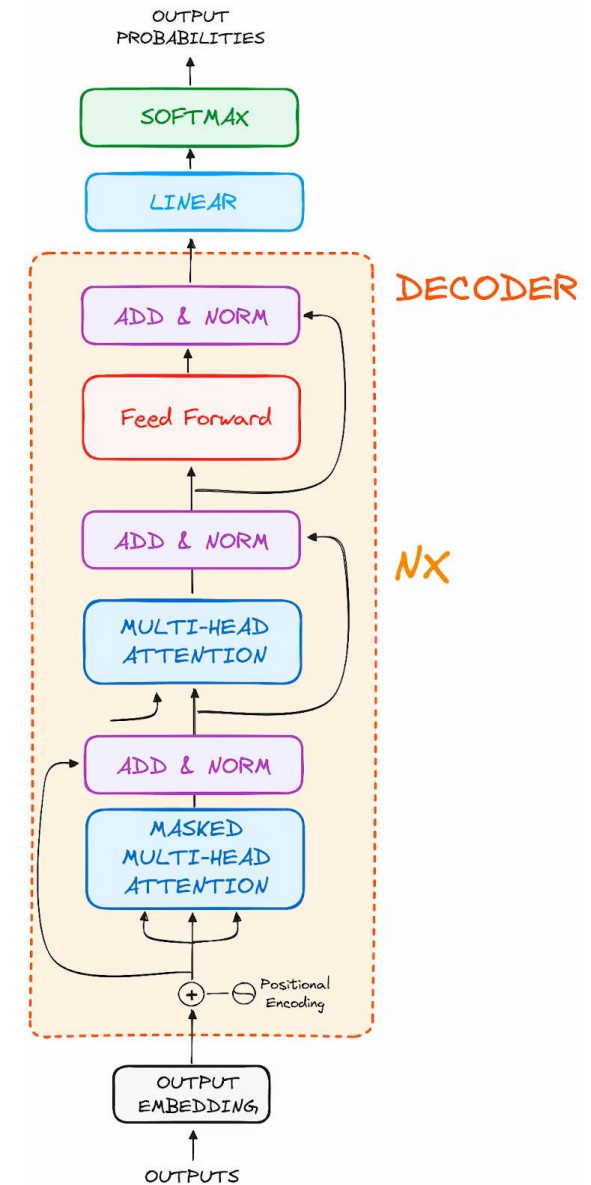


Imagen: [5]

Arquitectura de los Transformers

- ¿Qué hace el decodificador (*Decoder*)?
 - Mecanismo de autoatención enmascarada (*Masked Multi-Head Attention*) El decodificador solo tiene que prestar atención a la

El decodificador solo tiene que prestar atención a la palabra generada ahora y a las anteriores, **pero no a las futuras.**

Por ejemplo:

- Secuencia original: “I love Italian food”.
- Secuencia traducida: “Amo la comida italiana”.
- Si nos ubicamos en la palabra “la”, el decodificador solo tendrá acceso a la palabra “amo”.

Enmascarar: poner con valor 0 las palabras a las que no se debe de prestar atención.



Imágenes: [4]

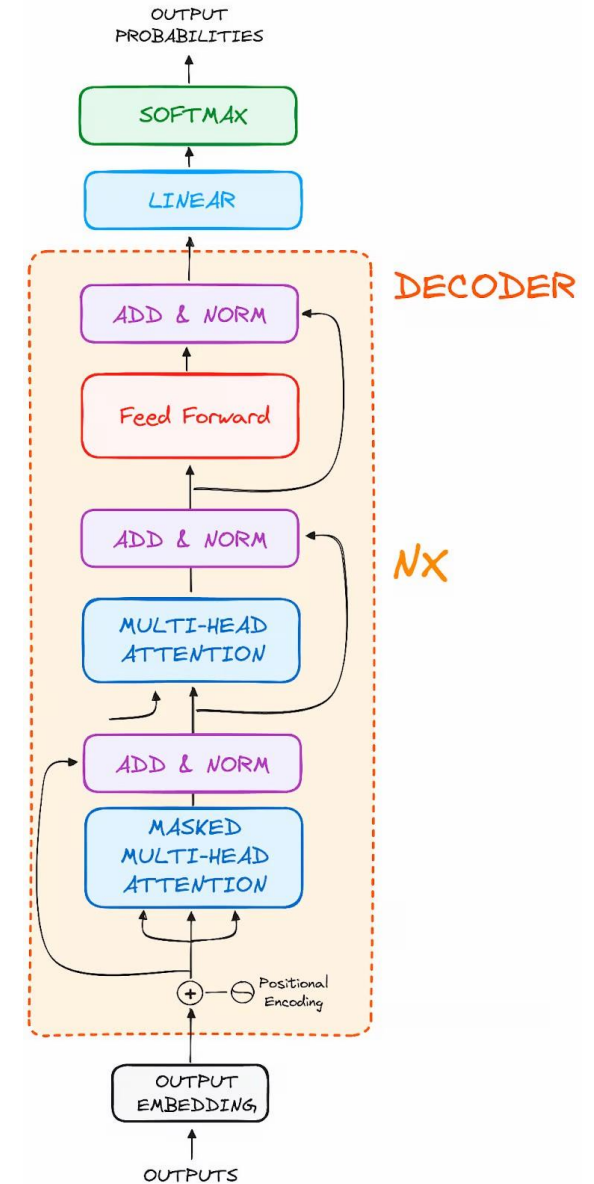


Imagen: [5]

Arquitectura de los Transformers

- ¿Qué hace el descodificador (*Decoder*)?
 - *Multi-Head Attention*. Aquí interaccionan los componentes del codificador con el descodificador.

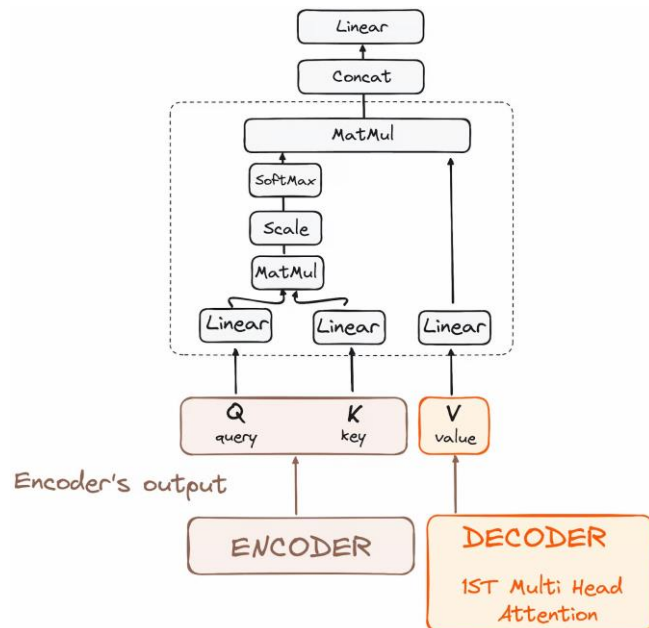


Imagen: [5]

En esta capa se enfoca la atención tanto en la secuencia original como en la salida.

La salida del codificador asume los papeles de consultas y claves.

Los valores provienen de la salida del descodificador (el dato proveniente de la capa residual).

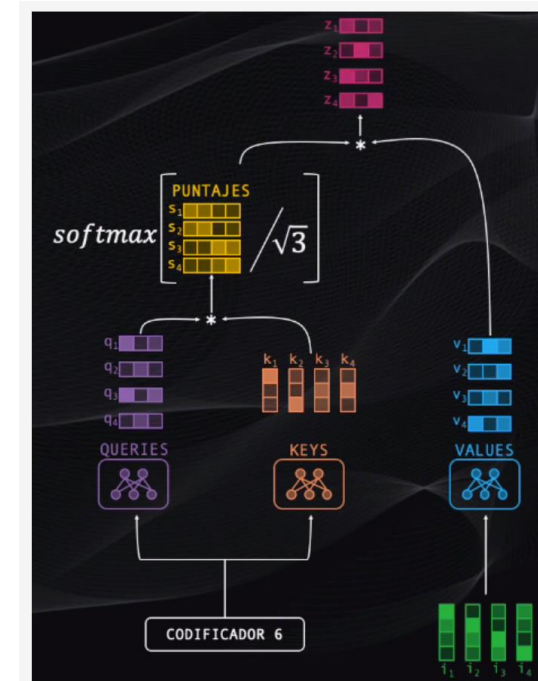


Imagen: [4]

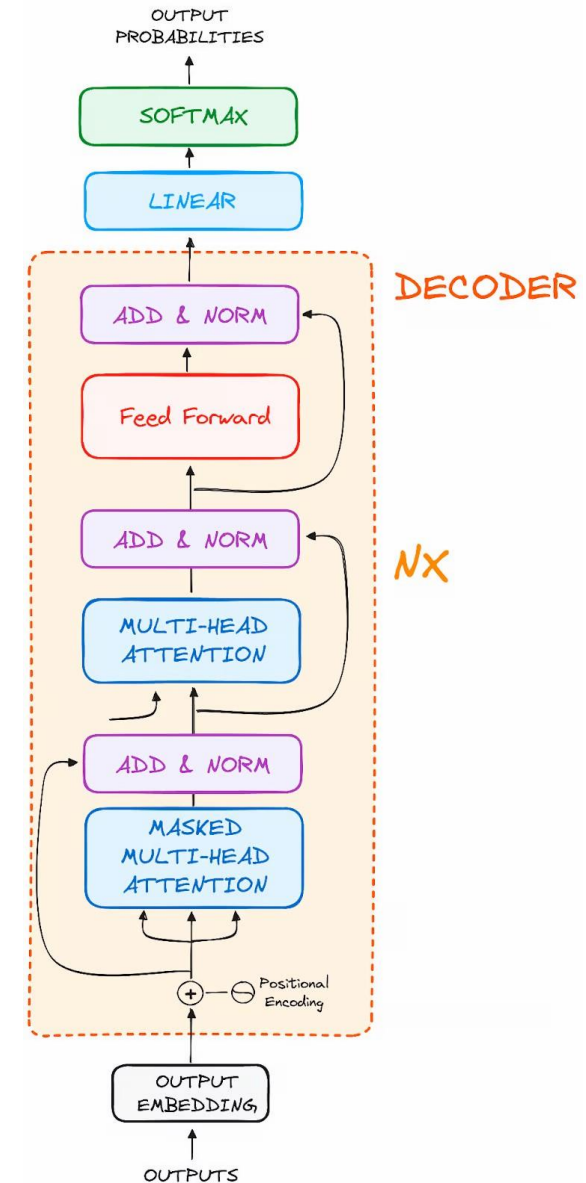


Imagen: [5]

Arquitectura de los Transformers

- ¿Qué hace el decodificador (*Decoder*)?
 - *Multi-Head Attention*. Aquí interaccionan los componentes del codificador con el decodificador.

De nuevo, se usan múltiples bloques atencionales de manera simultánea para codificar asociaciones a diferentes niveles.



En la etapa de decodificación de una Red Transformer se requieren múltiples decodificadores para encontrar relaciones entre palabras y grupos de palabras a diferentes niveles

Imagen: [4]

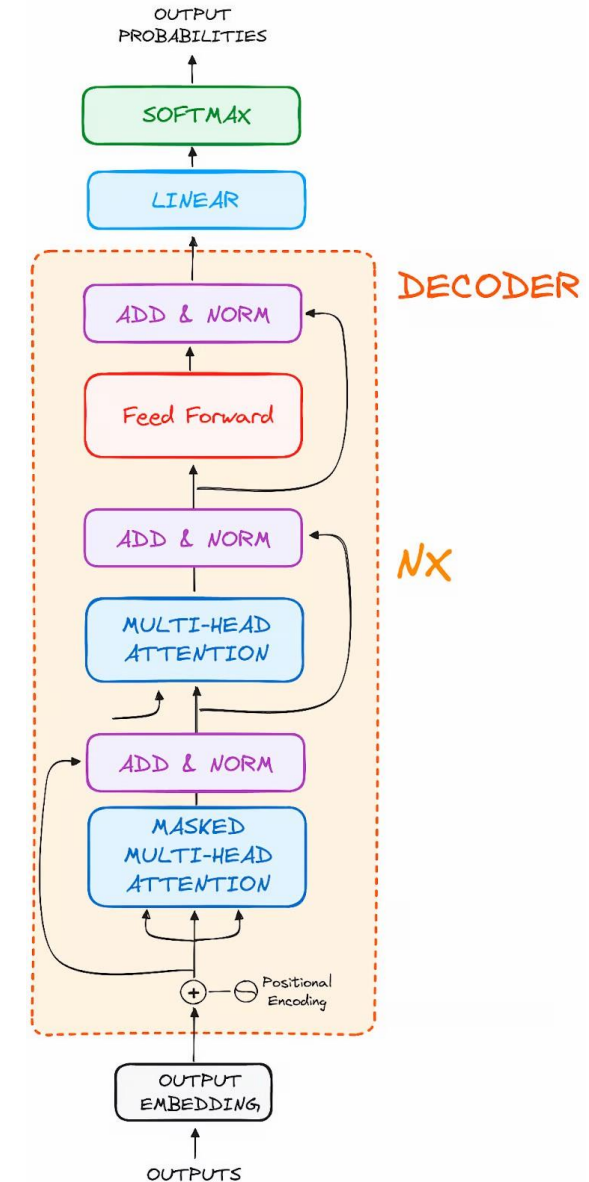
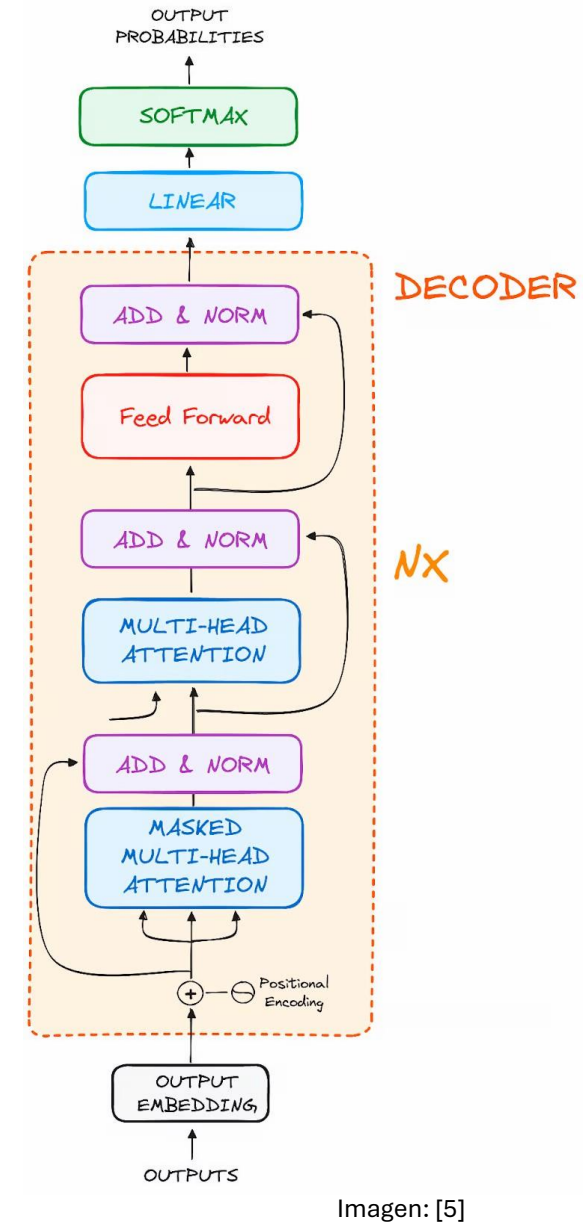
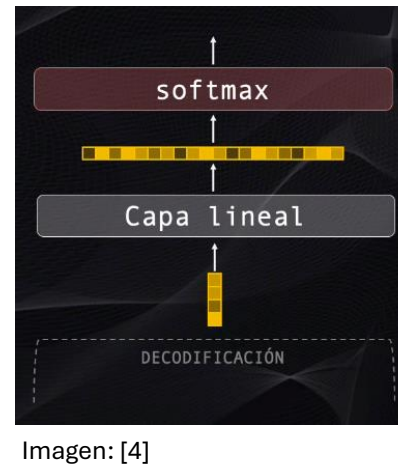
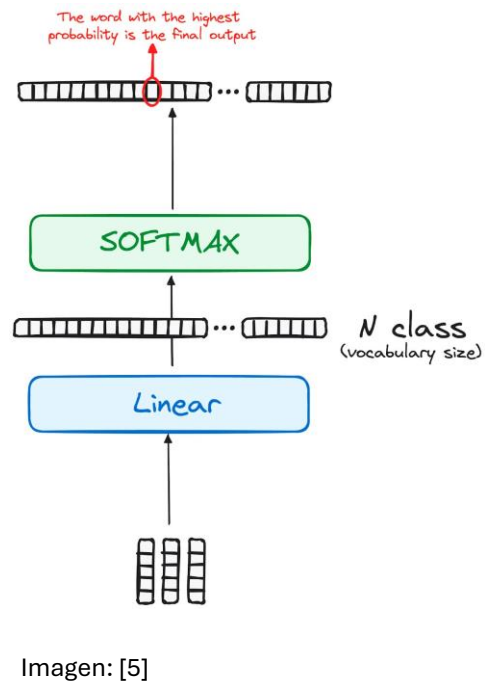


Imagen: [5]

Arquitectura de los Transformers

- ¿Qué hace el decodificador (*Decoder*)?
 - Clasificador línea y Softmax



Arquitectura de los Transformers

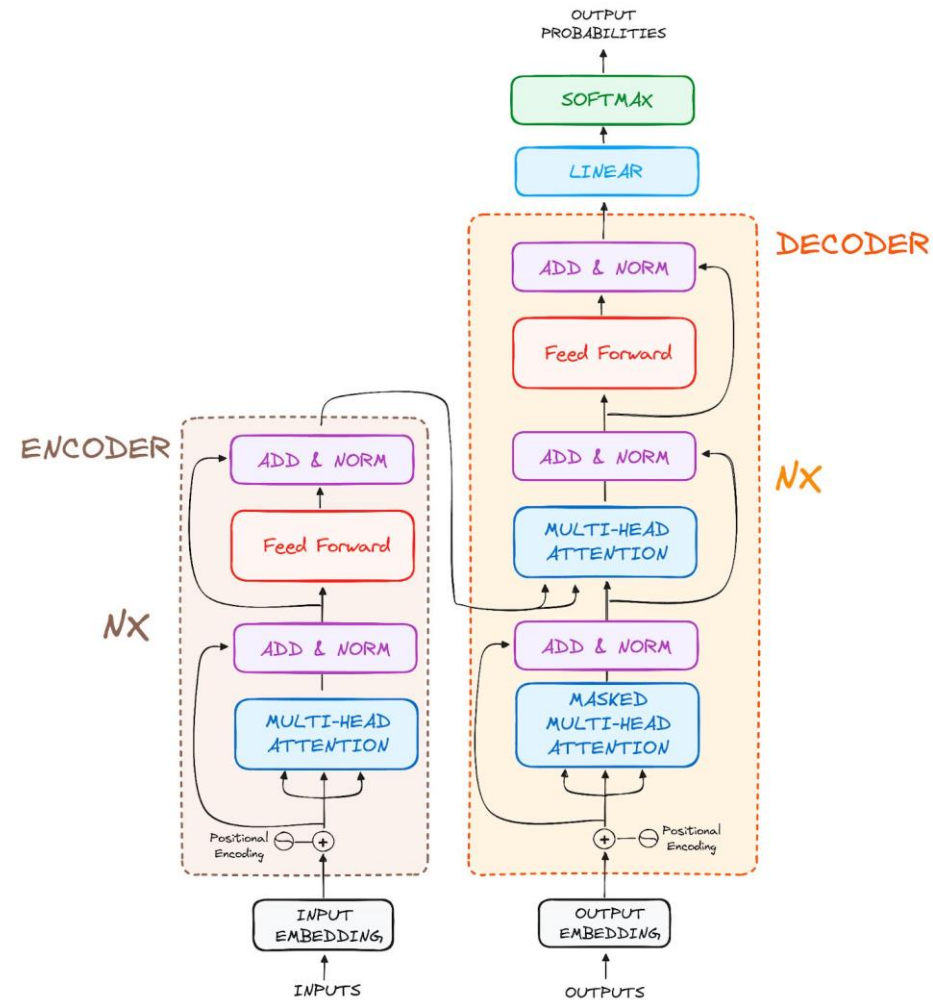


Imagen: [5]

Índice

- Conceptos preliminares
- ¿Qué son los Transformers?
- Arquitectura de los Transformers
- **¿Cómo se trabaja con los Transformers?**
- Limitaciones y desafíos de los Transformers en PLN
- Bibliografía

¿Cómo se trabaja con los Transformers?

- De forma general, se suele trabajar con Transformers en dos fases:
 1. Pre-training: En esta fase, el modelo aprende **cómo se estructura el lenguaje** de forma general y consigue un conocimiento genérico del significado de las palabras.
 2. Fine-tuning: se añaden ciertas capas a la arquitectura para **adaptar los modelos a tareas concretas** => se pre-entrena el modelo para esas tareas.
 - Se ajusta un Transformer preentrenado para una tarea concreta utilizando un dataset determinado.
 - El modelo preentrenado se entrenó sobre una gran colección de textos => la tarea de fine-tuning necesita menos ejemplos, el dataset puede ser mucho más pequeño.
 - Se extiende el transformer añadiendo nuevas capas. Y todo (modelo preentrenado y las nuevas capas) se entrena con el conjunto de datos de entrenamiento de la tarea que se vaya a abordar.
 - Ejemplos de fine-tuning:
 - Tarea de clasificación de textos => capa de softmax sobre la salida del modelo para predecir la clase para el texto de entrada.
 - NER (*Named Entity Recognition*) => se añade un CRF para predecir la etiqueta de cada token.

Índice

- Conceptos preliminares
- ¿Qué son los Transformers?
- Arquitectura de los Transformers
- ¿Cómo se trabaja con los Transformers?
- **Limitaciones y desafíos de los Transformers en PLN**
- Bibliografía

Limitaciones y desafíos de los Transformers en PLN

- Posibles limitaciones:
 - Requerimientos computacionales.
 - Sesgo y calidad de los datos de entrenamiento.
 - Capacidad de generalización.
 - Interpretabilidad y explicabilidad.

Índice

- Conceptos preliminares
- ¿Qué son los Transformers?
- Arquitectura de los Transformers
- ¿Cómo se trabaja con los Transformers?
- Limitaciones y desafíos de los Transformers en PLN
- **Bibliografía**

Bibliografía

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I. Attention Is All You Need. arXiv, 2023. ([Enlace](#))
- [2] Segura Bedmar, I. Transformers. Procesamiento del Lenguaje Natural con Aprendizaje Profundo. Máster en Ciencia y Tecnología Informática. 2023. ([Enlace](#))
- [3] Vaswani, A. et al. Attention is all you need. Advances in neural information processing systems, 30, 2017.
- [4] Codificandobits. Redes Transformers. ([Enlace](#))
- [5] Ferrer, J. Cómo funcionan los transformadores: Una exploración detallada de la arquitectura de los transformadores, 2024. ([Enlace](#))
- [6] Zidane, A. Positional Encoding, 2023. ([Enlace](#))
- [7] Awofeso, Z. “Attention Is All You Need” Explained, 2025. ([Enlace](#))